

Целочисленное линейное программирование. Метод ветвей и границ

Цель: изучить постановку задачи целочисленного линейного программирования (ЦЛП); получить навыки решения задачи ЦЛП методом ветвей и границ (МВГ).

Вариант №10

Найти целочисленное решение $\mathbf{x} = (x_1, x_2, x_3)^T$ следующей задачи:

$$F = \mathbf{c}\mathbf{x} \rightarrow \max;$$

$$\mathbf{A}\mathbf{x} \leq \mathbf{b};$$

$$x_1, x_2, x_3 \geq 0.$$

$$\begin{aligned} \mathbf{c} &= (7 \ 5 \ 3) \\ \mathbf{A} &= \begin{pmatrix} 4 & 1 & 1 \\ 1 & 2 & 0 \\ 0 & 0,5 & 1 \end{pmatrix} \\ \mathbf{b}^T &= (4 \ 3 \ 2) \end{aligned}$$

Результаты работы:

Решение задачи ЦЛП методом полного перебора

```
# Полный перебор
best_F = -1
best_x = None

for x1 in range(0, 2): # максимум по x1 ограничен 4/4=1
    for x2 in range(0, 2): # максимум по x2 из второго ограничения
        for x3 in range(0, 3): # максимум по x3 из третьего ограничения
            if 4*x1 + x2 + x3 <= 4 and x1 + 2*x2 <= 3 and 0.5*x2 + x3 <= 2:
                F = 7*x1 + 5*x2 + 3*x3
                if F > best_F:
                    best_F = F
                    best_x = (x1, x2, x3)

print('Лучшее целочисленное решение:', best_x, 'F =', best_F)
```

✓ 0.0s

Python

Лучшее целочисленное решение: (0, 1, 1) F = 8

Рисунок 1 Результат перебора

```

Defaulting to user installation because normal site-packages is not writeable
Collecting pulp
  Using cached pulp-3.3.0-py3-none-any.whl.metadata (8.4 kB)
Using cached pulp-3.3.0-py3-none-any.whl (16.4 MB)
Installing collected packages: pulp
Successfully installed pulp-3.3.0
Welcome to the CBC MILP Solver
Version: 2.10.3
Build Date: Dec 15 2019

command line - /home/hronoz/.local/lib/python3.13/site-packages/pulp/apis/./solverdir/cbc/linux/i64/cbc /tmp/cc3cd44462e5445eb3a4035ec723122c-pulp.mps -n
At line 2 NAME          MODEL
At line 3 ROWS
At line 8 COLUMNS
At line 19 RHS
At line 23 BOUNDS
At line 24 ENDDATA
Problem MODEL has 3 rows, 3 columns and 7 elements
Coin0008I MODEL read with 0 errors
Option for timeMode changed from cpu to elapsed
Presolve 3 (0) rows, 3 (0) columns and 7 (0) elements
0 Obj -0 Dual inf 15 (3)
0 Obj -0 Dual inf 15 (3)
3 Obj 13
Optimal - objective value 13
...
Option for printingOptions changed from normal to all
Total time (CPU seconds):      0.00   (Wallclock seconds):      0.00

Решение ЛП: (0.33333333, 1.33333333, 1.33333333) F = 12.99999971
Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...

```

Рисунок 2 Симплекс

Сравнение целочисленного решения с решением ЛП

[Generate](#)
[+ Code](#)
[+ Markdown](#)

```

import numpy as np

# Округление LP решения вниз
lp_floor = tuple(int(np.floor(v)) for v in lp_solution)
F_floor = 7*lp_floor[0] + 5*lp_floor[1] + 3*lp_floor[2]

# Округление LP решения вверх
lp_ceil = tuple(int(np.ceil(v)) for v in lp_solution)
F_ceil = 7*lp_ceil[0] + 5*lp_ceil[1] + 3*lp_ceil[2]

print('Округление вниз:', lp_floor, 'F =', F_floor)
print('Округление вверх:', lp_ceil, 'F =', F_ceil)
print('Лучшее ЦЛП решение:', best_x, 'F =', best_F)

```

✓ 0.0s Python

Округление вниз: (0, 1, 1) F = 8
Округление вверх: (1, 2, 2) F = 23
Лучшее ЦЛП решение: (0, 1, 1) F = 8

Рисунок 3 Итог

Листинг программы:

```

# %% [markdown]
# # Отчет по работе: Целочисленное линейное программирование
#
# **Студент:** Иванов И.И.
#
# **Группа:** 101

```

```

#
# Дата: 04.11.2025

# %% [markdown]
# ## Цель работы
# Изучение методов решения задач целочисленного линейного программирования
(ЦЛП) и сравнение с классическим ЛП.

# %% [markdown]
# ## Постановка задачи
# Максимизировать функцию:
#  $F = 7x_1 + 5x_2 + 3x_3 \rightarrow \max$ 
# при ограничениях:
#  $4x_1 + x_2 + x_3 \leq 4$ 
#  $x_1 + 2x_2 \leq 3$ 
#  $0.5x_2 + x_3 \leq 2$ 
# и неотрицательных целых переменных  $x_1, x_2, x_3 \geq 0$ 
#

# %% [markdown]
# ## Решение задачи ЦЛП методом полного перебора

# %%
# Полный перебор
best_F = -1
best_x = None

for x1 in range(0, 2): # максимум по x1 ограничен 4/4=1
    for x2 in range(0, 2): # максимум по x2 из второго ограничения
        for x3 in range(0, 3): # максимум по x3 из третьего ограничения
            if 4*x1 + x2 + x3 <= 4 and x1 + 2*x2 <= 3 and 0.5*x2 + x3 <= 2:
                F = 7*x1 + 5*x2 + 3*x3
                if F > best_F:
                    best_F = F
                    best_x = (x1, x2, x3)

print('Лучшее целочисленное решение:', best_x, 'F =', best_F)

# %% [markdown]
# ## Решение исходной задачи ЛП симплекс-методом

# %%
import sys
!{sys.executable} -m pip install pulp
import pulp

# ЛП задача
prob = pulp.LpProblem('LP_Maximization', pulp.LpMaximize)
x1 = pulp.LpVariable('x1', lowBound=0)
x2 = pulp.LpVariable('x2', lowBound=0)
x3 = pulp.LpVariable('x3', lowBound=0)

prob += 7*x1 + 5*x2 + 3*x3, 'F'
prob += 4*x1 + x2 + x3 <= 4
prob += x1 + 2*x2 <= 3
prob += 0.5*x2 + x3 <= 2

prob.solve()

lp_solution = (x1.value(), x2.value(), x3.value())
lp_F = pulp.value(prob.objective)
print('Решение ЛП:', lp_solution, 'F =', lp_F)

# %% [markdown]

```

```
# ## Сравнение целочисленного решения с решением ЛП

# %%
import numpy as np

# Округление LP решения вниз
lp_floor = tuple(int(np.floor(v)) for v in lp_solution)
F_floor = 7*lp_floor[0] + 5*lp_floor[1] + 3*lp_floor[2]

# Округление LP решения вверх
lp_ceil = tuple(int(np.ceil(v)) for v in lp_solution)
F_ceil = 7*lp_ceil[0] + 5*lp_ceil[1] + 3*lp_ceil[2]

print('Округление вниз:', lp_floor, 'F =', F_floor)
print('Округление вверх:', lp_ceil, 'F =', F_ceil)
print('Лучшее ЦЛП решение:', best_x, 'F =', best_F)
```

Вывод: в ходе выполнения лабораторной работы была поставлена и решена задача целочисленного линейного программирования по МВГ.